

Tarea 3: Roles y Privilegios

Orientado a PostgreSQL

Bases de Datos

1 Concepto de roles y privilegios

Definición

Un **rol** es la entidad que en PostgreSQL representa, de forma unificada, tanto a un usuario como a un grupo; agrupa privilegios y, si tiene el atributo **LOGIN**, también credenciales de conexión. Un **privilegio** es el permiso otorgado a un rol para ejecutar una operación puntual sobre un objeto de la base de datos o sobre el sistema en su conjunto.

Explicación

Desde la versión 8.1, PostgreSQL no distingue entre “usuarios” y “grupos” como conceptos separados: todo es un rol, y lo que cambia es la combinación de atributos que tiene activada. Los privilegios que puede ejercer un rol se agrupan en dos grandes categorías (detalladas en la sección 3): los que actúan sobre el sistema en general (por ejemplo, poder conectarse o crear bases de datos) y los que actúan sobre un objeto puntual (por ejemplo, poder leer una tabla). Un rol puede además ser miembro de otros roles y heredar sus privilegios, de forma similar a como una clase hereda de otra en programación orientada a objetos. En la práctica funciona igual que los permisos **rx** de un archivo en Linux, solo que aplicados a tablas, vistas y funciones en lugar de a archivos.

2 Cómo dar y quitar privilegios

Definición

GRANT es la sentencia que otorga uno o más privilegios sobre un objeto a un rol; **REVOKE** es la sentencia que los retira. Ambas siguen la misma sintaxis general, solo invirtiendo la dirección de la operación:

```
GRANT privilegio [, ...] ON objeto TO rol [WITH GRANT OPTION];
REVOKE privilegio [, ...] ON objeto FROM rol [CASCADE | RESTRICT];
```

Explicación

Solo el propietario del objeto (o un superusuario) puede ejecutar estas sentencias sobre él, salvo que el propietario haya delegado esa capacidad mediante **WITH GRANT OPTION**. Es equivalente a firmar un gafete de acceso: **GRANT** es dárselo a alguien, **REVOKE** es quitárselo. Si esa persona a su vez prestó su gafete a un tercero (porque tenía **GRANT OPTION**), revocar el original obliga a decidir si también se retira ese préstamo derivado (usando **CASCADE**) o si la operación se rechaza mientras existan esos préstamos pendientes (comportamiento **RESTRICT**, el que aplica por defecto).

```
GRANT SELECT, INSERT ON alumnos TO analista;
REVOKE INSERT ON alumnos FROM analista;
```

3 Privilegios a nivel sistema y objeto

Definición

Los **privilegios de sistema** (o atributos de rol) son capacidades globales del clúster asignadas con `CREATE ROLE` / `ALTER ROLE`, como `LOGIN`, `SUPERUSER`, `CREATEDB` o `CREATEROLE`. Los **privilegios de objeto** son permisos otorgados con `GRANT` sobre un elemento específico de una base de datos (tabla, vista, secuencia, función, esquema o columna), como `SELECT`, `INSERT` o `EXECUTE`.

Explicación

Un rol necesita ambos niveles para operar: por ejemplo, `LOGIN` (sistema) para conectarse, y `SELECT` sobre una tabla (objeto) para poder consultarla. La forma más clara de verlo es pensando en un edificio de oficinas. Los privilegios de sistema son la credencial que decide si entras al edificio en general: ¿puedes entrar de noche?, ¿puedes construir oficinas nuevas?, ¿eres el dueño del edificio (superusuario)? Los privilegios de objeto son lo que puedes hacer *dentro* de una oficina específica una vez que ya entraste: ¿puedes abrir el archivero (`SELECT`)?, ¿puedes meter papeles nuevos (`INSERT`)?, ¿puedes tirar documentos (`DELETE`)? Tener acceso al edificio no implica poder abrir cualquier archivero: son controles independientes.

4 Diferencia entre role y usuario

Definición

En PostgreSQL no existe una entidad “usuario” distinta del rol: `CREATE USER` es un alias de `CREATE ROLE` que únicamente cambia el valor por defecto del atributo `LOGIN` (lo activa), mientras que `CREATE ROLE` lo deja desactivado (`NOLOGIN`) por defecto.

Explicación

En la práctica, un “usuario” es simplemente un rol con `LOGIN`, y un “grupo” es un rol sin `LOGIN` que funciona como contenedor de privilegios al que se agregan otros roles como miembros. No son dos cosas distintas: son el mismo molde usado de dos formas, según qué bandera se active.

```
CREATE ROLE analista LOGIN PASSWORD 'clave'; -- funciona como "usuario"
CREATE ROLE lectores_solamente NOLOGIN; -- funciona como "grupo"
GRANT lectores_solamente TO analista; -- analista hereda esos privilegios
```

5 Reglas importantes

- El propietario de un objeto tiene automáticamente todos los privilegios sobre él; no es necesario otorgárselos explícitamente.
- Los privilegios efectivos de un rol son la suma de: los otorgados directamente, los heredados de los roles de los que es miembro (si tiene `INHERIT`), y los otorgados a `PUBLIC` (el grupo implícito que incluye a todos los roles).
- Solo puede revocar un privilegio quien lo otorgó directamente, o quien es dueño del objeto / posee `GRANT OPTION` sobre él.
- Revocar un privilegio sobre una tabla revoca automáticamente los privilegios de columna correspondientes; lo inverso no ocurre.
- `CREATEROLE` no permite crear ni alterar roles `SUPERUSER`, ni otorgar/revocar `REPLICATION` o `BYPASSRLS`.

6 Consideraciones prácticas

- Evitar usar el rol `postgres` (superusuario) para las operaciones cotidianas de una aplicación; crear roles específicos con el mínimo privilegio necesario.
- Usar `ALTER DEFAULT PRIVILEGES` para que los objetos creados a futuro en un esquema hereden automáticamente un conjunto de privilegios, sin tener que otorgarlos tabla por tabla.
- Usar `\du` en `psql`, o consultar `pg_roles / information_schema.role_table_grants`, para auditar qué privilegios tiene cada rol.
- Definir `CONNECTION LIMIT` y `VALID UNTIL` en roles de aplicación o de acceso temporal, como medida adicional de seguridad.

Fuentes: Documentación oficial de PostgreSQL: Privileges (Cap. 5.8), GRANT, REVOKE, Role Attributes (Cap. 21.2), CREATE USER (postgresql.org/docs).